Step 4 — A Fallback When a Server Is Unreachable, and a Full Working Demo — Solution

> Back to the step · Hint 1 · Hint 2

Story: every step so far quietly assumed every server works. Production doesn't work that way. A server can be down for a deploy, misconfigured, or just slow — and an agent that can't tell "one tool source is gone" from "everything is broken" isn't trustworthy enough to run unattended. This step makes an unreachable server a logged-and-skipped non-event rather than a crash, a mid-task tool failure a clear observation the model can react to, and a runaway loop a named, catchable error. **If not:** one missing server would hang or crash the whole agent on the exact day you're demoing it.

This is the **final** version of `server_pool.py`, `main.py`, and `test_agent.py`.

`src/server_pool.py` (final)

Story: `connect_all` now puts a timeout on each server's handshake and wraps the whole attempt in `try/except` — a failed server is logged and skipped, while every other server still connects normally. `call_tool` wraps its own dispatch too, so a server that drops mid-task returns a clear `Error:` string the model can read, instead of crashing the whole agent loop. **If not:** a single unreachable or flaky server would take the whole agent down with it.

```
# MCPServerPool: connect to every configured server, discover and namespace
# its tools, and route calls back to the right server. Resilient: a server
# that can't connect is skipped; a tool call that fails mid-task returns a
# clear error string instead of crashing the loop.
import asyncio
from contextlib import AsyncExitStack

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

from src.tool_bridge import extract_text, mcp_tools_to_openai_schema
from src.utils.logger import get_logger

class MCPServerPool:
    def __init__(self, servers, connect_timeout=10.0):
```

```
        self.servers = servers
        self.connect_timeout = connect_timeout
        self.stack = AsyncExitStack()
        self.sessions = {}
        self.routing = {}
        self.named_tools = []
        self.logger = get_logger(__name__)

    async def _connect_one(self, server):
        # The context managers are entered in THIS (the main) task, never
        # inside asyncio.wait_for: wait_for runs its coroutine in a new
        # task, and a context entered in one task cannot be closed in
        # another (it raises "cancel scope in a different task" at
        # teardown). Only initialize() -- the call that can hang -- is
        # wrapped in the timeout.
        params = StdioServerParameters(
            command=server["command"], args=server["args"]
        )
        read, write = await self.stack.enter_async_context(
            stdio_client(params)
        )
        session = await self.stack.enter_async_context(
            ClientSession(read, write)
        )
        await asyncio.wait_for(
            session.initialize(), timeout=self.connect_timeout
        )
        return session

    async def connect_all(self):
        for server in self.servers:
            name = server["name"]
            try:
                session = await self._connect_one(server)
            except Exception as e:
                self.logger.warning(
                    f"[{name}] unreachable: {e}, continuing without it"
                )
                continue
            self.sessions[name] = session
            listed = await session.list_tools()
            for tool in listed.tools:
                namespaced = name + "__" + tool.name
                self.routing[namespaced] = (name, tool.name)
                self.named_tools.append((namespaced, tool))
            self.logger.info(f"[{name}] connected, {len(listed.tools)} tools")
```

```
    def openai_tools(self):
        return mcp_tools_to_openai_schema(self.named_tools)

    async def call_tool(self, namespaced_name, arguments):
        if namespaced_name not in self.routing:
            return f"Error: no tool named {namespaced_name} is available"
        server_name, tool_name = self.routing[namespaced_name]
        session = self.sessions[server_name]
        try:
            result = await session.call_tool(tool_name, arguments)
        except Exception as e:
            self.logger.warning(f"[{server_name}] call failed: {e}")
            return f"Error: tool {namespaced_name} failed: {e}"
        return extract_text(result)

    async def aclose(self):
        await self.stack.aclose()
```

> **The one real async trap, and why it's worth a comment in the code:** your first instinct is to write `await asyncio.wait_for(self._connect_one(server), timeout=...)` — timing out the whole connection. Don't. `asyncio.wait_for` runs its coroutine in a **new task**, so the `stdio_client` / `ClientSession` context managers get *entered* in that task. When `aclose()` later tries to *exit* them in the main task, anyio raises `RuntimeError: Attempted to exit cancel scope in a different task than it was entered in`. Enter the contexts in the main task and time out only `initialize()`, which is the call that actually hangs when a server never finishes starting.

`run_agent` already raises `MaxIterationsExceeded` (written in Step 2) — that's the loop's hard limit, now a named error instead of a silent stop, so a model that keeps retrying a failing call can't spin forever.

`main.py` (final demo)

Story: this ties the whole project together in one runnable demo — connect to the configured servers (with one deliberately broken), show exactly what connected and what didn't, run a task that needs both working servers, and finish cleanly either way. **If not:** you'd have no single command that proves, end to end, that the agent degrades gracefully instead of just claiming to.

```
# Final demo: connect to whatever servers are reachable (one is
# deliberately broken here), discover their tools, run a real multi-tool
# task, and finish cleanly whether or not every server came up.
import asyncio
```

```
import sys

from src.agent import run_agent
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import create_client
from src.server_pool import MCPServerPool
from src.servers_config import SERVERS
from src.utils.logger import get_logger

TASK = ("Check my notes for anything about the deploy process, and also "
        "list the files in the project folder.")

async def run_demo():
    logger = get_logger(__name__)
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    client = create_client(config)
    pool = MCPServerPool(SERVERS, connect_timeout=config.connect_timeout)
    await pool.connect_all()
    print(f"Connected servers: {list(pool.sessions.keys())}")
    print(f"Tools discovered: {len(pool.named_tools)}")

    try:
        print(f"\nTask: {TASK}")
        result = await run_agent(
            TASK, pool, client, max_iterations=config.max_iterations
        )
        for step in result.steps:
            print(f"  -> {step['tool']} {step['args']}")
        print(f"Answer: {result.answer}")
    finally:
        await pool.aclose()

def main():
    asyncio.run(run_demo())

if __name__ == "__main__":
    main()
```

To see the fallback live, add a broken server to `src/servers_config.py` (a script that doesn't exist) — the demo logs it and keeps going:

```
Connected servers: ['notes', 'filesystem']
Tools discovered: 4
...
```

With the notes server itself broken, the line becomes `Connected servers: ['filesystem']` and the agent answers using only the filesystem tools — no hang, no crash.

`tests/integration/test_agent.py` (final — add case 4)

Story: this is the resilience proof — give the pool one deliberately broken server alongside one good one, and confirm the broken one is skipped while the agent still runs normally on what's left. **If not:** "it degrades gracefully" would be a claim you never actually tested against a real failure.

Add the broken server and this case:

```
BROKEN = {"name": "broken", "command": "python",
          "args": ["servers/does_not_exist.py"]}

async def test_fallback_when_server_down():
    config, client, pool = await connect([NOTES, BROKEN])
    result = "OK"
    if "broken" in pool.sessions:
        result = "WRONG"
    if "notes" not in pool.sessions:
        result = "WRONG"
    out = await run_agent("What does my note about the Q3 roadmap say?",
                          pool, client, config.max_iterations)
    if not out.answer:
        result = "WRONG"
    await pool.aclose()
    print(f"case 4 (broken server skipped)         -> {result}")
```

and call it from `main()`:

```
    await test_fallback_when_server_down()
```

Expected output (`python -m tests.integration.test_agent`) — all four cases, against real MCP servers:

```
case 1 (tools discovered + namespaced) -> OK
case 2 (chained search then read)      -> OK
```

```
case 3 (routes to both servers)        -> OK
case 4 (broken server skipped)         -> OK
```

(The broken server logs one `WARNING` line — `[broken] unreachable: …, continuing without it` — on stderr, then the run proceeds on the notes server alone.)

Done

You now have a single ReAct agent that: - discovers every tool at runtime from live MCP servers — none written into its own code, - connects to several servers at once, namespacing and routing each call to the right one, - keeps working when a server is unreachable or drops mid-task, and - stops cleanly at a hard step limit instead of looping forever.

That's the consumer side of a real protocol, with the resilience a real deployment needs — exactly the thing a hardcoded tool list can never give you.